

ИССЛЕДОВАНИЕ

15 ОКТЯБРЯ 2025 ГОДА

Итеративный SFT (iSFT): обучение с плотным вознаграждением

Итеративный SFT: обучение с высокой пропускной способностью



Чарльз О'Нил (Базеттен)



Джонатон Лю (Бастетен)



Гарри Партридж (Бастетен)



Макс Киркби (Baseten)



Мудит Джаясекара (Басетен)

Проблема тонкой настройки

Если вы хотите, чтобы модель выполняла очень специфическую задачу, например следовала определенной политике, соблюдала требования интерфейса инструмента, оперировала данными в определенном формате, соблюдала требования безопасности или бизнес-ограничения, проще всего обучить ее на контролируемых примерах желаемого поведения. Это стандартная контролируемая дообучение-на-примерах (supervised fine-tuning, SFT): соберите пары (входные данные → идеальный результат) и обучите модель имитировать эти результаты с помощью прогнозирования следующего токена.

Однако в большинстве реалистичных сценариев у вас на самом деле нет идеальных результатов для SFT. Возможно, у вас есть базовая модель, которая может справиться с задачей, но делает это довольно плохо, или у нее есть предел производительности, который, как вы знаете, вам никогда не преодолеть. Возможно, у вас есть инструмент для оценки (например, LLM в роли судьи или верификатора), с помощью которого вы можете определить, хороший ответ или плохой. Но у вас *нет* чистого набора данных с идеальными примерами для подражания. Это распространенная проблема, из-за которой обычный SFT перестает работать.

Несмотря на то, что тонкая настройка с помощью дистилляции может помочь (SFT на основе данных, сгенерированных мощной обучающей моделью), она может оказаться неэффективной, если даже самые лучшие модели не справляются с вашей конкретной задачей. В таком случае дистиллированные результаты не являются подходящими целевыми показателями для имитации. А даже если и являются, вы ограничены возможностями обучающей модели. Даже при наличии неограниченного количества данных вы сможете достичь только того же уровня производительности, что и обучающая модель.

Обычно в качестве следующего шага люди пробуют использовать оптимизацию в стиле обучения с подкреплением. «Если я могу оценить ответ, то могу поощрять модель за правильные ответы и наказывать за неправильные». Это работает, но не без подводных камней. В частности, вознаграждение выдается нерегулярно. Если текущая модель выдает в основном неправильные ответы, вы просто получаете «плохое» вознаграждение, и у нее нет стимула для обучения.

Итак, мы застряли между двух вариантов:

- **SFT**: нужны данные о золоте, которых у нас нет.
- **RL**: данные о золоте не нужны, но этот вариант тяжелее и менее надежный, чем тот, который мы хотели бы использовать на первом этапе.

Нам нужно что-то среднее:

- Так же просто/стабильно, как SFT.
- Так же масштабируемо, как RL (без участия человека в каждом примере).
- И соответствует *нашему* определению задачи, а не просто «тому, что сказал бы GPT-5».

Вот что дает нам итеративная SFT.

Итеративная SFT

Ключевой момент заключается в том, что, даже не зная, как мгновенно написать идеальный ответ, мы часто можем определить, что ответ неверный, и подсказать, как его исправить. Для многих задач у нас уже есть:

- оценщик («проверяющий»), который может сказать, является ли ответ приемлемым, и
- какой-то способ вносить правки в зависимости от отзывов.

Итеративная SFT преобразует их в обучающие данные. Процесс, шаг за шагом:

1. **Сгенерировать:** начните с входных данных x и заставьте текущую модель выдать результат y_0 . Первые попытки могут оказаться неудачными.
2. **Оценка и пояснения:** Запустите проверку на y_0 . Система проверки (LLM в роли судьи) не просто выдает результат «зачтено» или «не зачтено», но и может объяснить, *почему* оценка неудовлетворительная («в цепочке рассуждений пропущен шаг», «вы использовали неправильный инструмент», «неверный формат» и т. д.). Теперь у вас есть структурированная обратная связь.
3. **Доработка:** Используйте эту обратную связь, чтобы получить новый результат y_1 . Переоцените. Если результат все еще не идеален, повторите: $y_2, y_3, \dots$
4. **Прекратить, когда результат будет идеальным:** Как только мы получим ответ y^* , который оценщик сочтет полностью правильным / соответствующим правилам / соответствующим формату, мы сохраним его.

Теперь мы создали чистую контролируемую точку данных $x \rightarrow y^*$.

Если проделать то же самое с большим набором входных данных, то можно получить набор данных качества SFT **без использования эталонных ответов, написанных людьми**. Плохие результаты работы модели были исправлены еще до начала обучения.

Мы называем этот процесс итеративной SFT (iSFT).¹

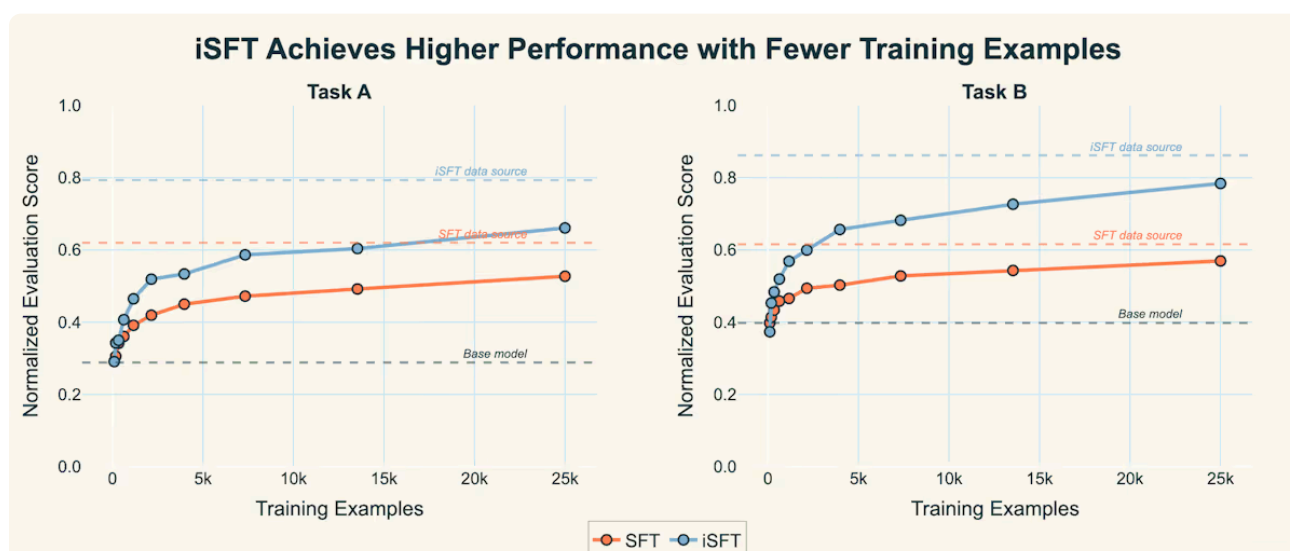
Обратите внимание, что для эффективной работы этого процесса нам нужен очень надежный и полный набор оценочных инструментов. В противном случае полученные в итоге точки данных y^* могут оказаться некачественными, и вы будете выполнять тонкую настройку на зашумленных или неполных данных (оценочный инструмент может противоречить сам себе, быть непоследовательным или неполным). К счастью, у нас есть процесс получения таких оценочных инструментов. Обычно мы полагаемся на мощные модели, такие как **gemini-2.5-pro** с максимальной степенью проработанности, которые выступают в роли LLM-судьи.

By repairing model attempts until they pass the evaluator you actually care about, iSFT can produce training sequences that are higher quality than the sequences generated through pure distillation. This means we can actually outperform any of the closed-source models we care to start with. Here, we're using our maximally intelligent model to be the grader rather than to generate the answers from scratch, which is a simpler task.

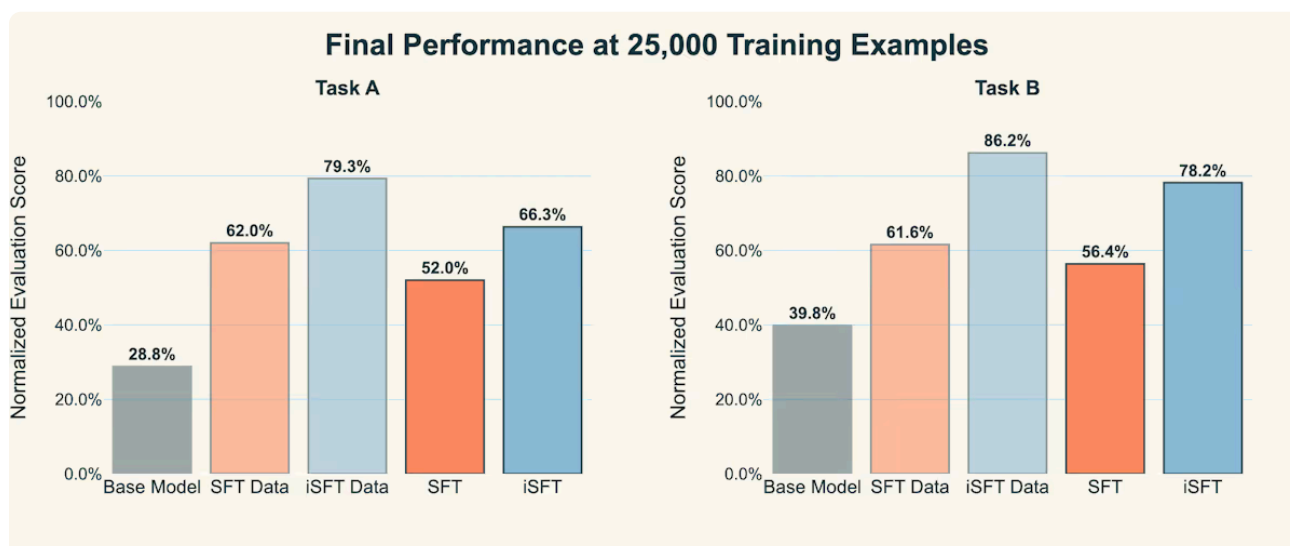
Iterative SFT works

We evaluated iSFT against SFTing on the outputs of gemini-2.5-pro for two real-world medical scribe tasks in active production use: dental clinical notes (Task A) and emergency department notes (Task B). Both tasks use custom graders developed over several weeks with domain experts, scoring outputs on 6-point and 5-point scales respectively. All experiments use **gemini-2.5-pro** for data generation, evaluation and refinement, and **qwen3-32b** as the model we're training. We train with LoRA adapters, rank 256, with alpha=32, a constant learning rate ($lr=2e-4$), and zero LoRA dropout.

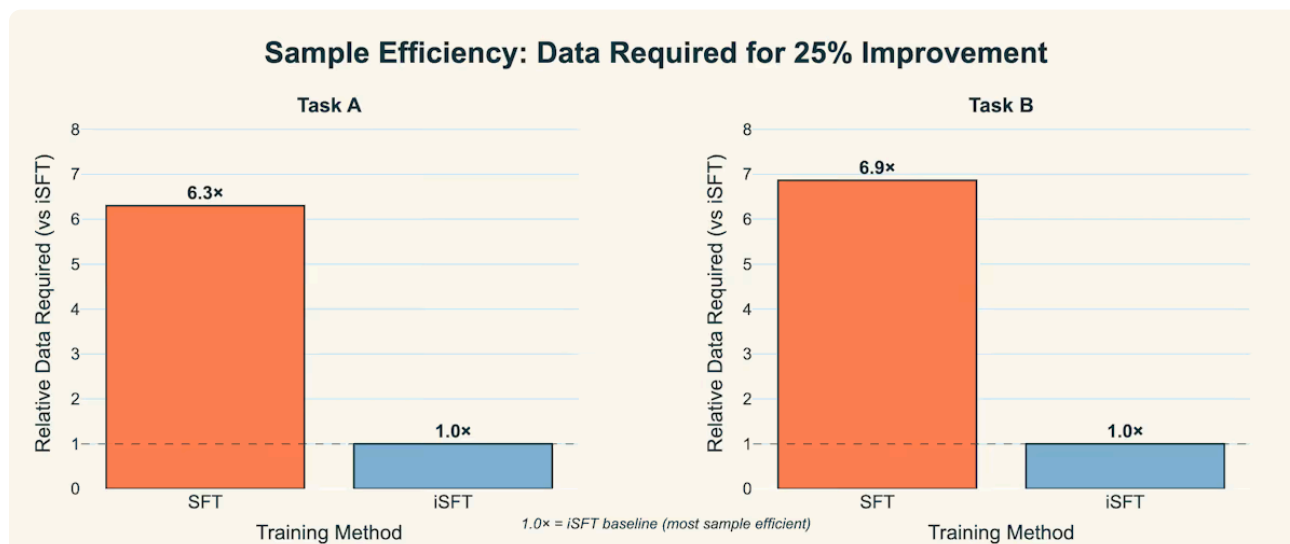
At 25,000 training examples, each method plateaus at distinct performance levels: standard SFT achieves 52-56% normalized accuracy, whereas iterative SFT reaches 66-78%, and RGT climbs to 72-83% across both tasks.



Put more clearly, the scores of the distribution of data we're training on for iSFT is much higher than the plain SFT data. The actual algorithm that learns the gradient update is the same; the difference is just the quality of data we're training on.



We can also look at this from a sample efficiency perspective. In terms of actual number of examples we'd need to train on in order to reach a 25% improvement in score, plain SFT requires around 6-7 times more data. We fitted logarithmic growth models to the learning curves and used binary search with linear interpolation to precisely identify where each method crosses this threshold.



Note, this isn't an entirely fair comparison because we use a lot of tokens in producing the high-quality outputs for iSFT. But it goes to show that if you care about the ceiling of performance, you can get much higher than even the base models with this non-RL approach.

To summarize, iSFT scales monotonically. More data just makes you better.

Because all training pairs are already “perfect” under the grader, you don't hit the usual plateau where adding more noisy data starts to hurt. You can just generate → refine → absorb into the dataset, over and over. This works with normal supervised training infrastructure: no custom RL loop, no special credit assignment tricks, no weird unstable hyperparameters. You can drop the resulting dataset into any standard SFT pipeline and warm-start a model that's already close to doing the task, before you do any RL fine-tuning.

iSFT vs RL: the information bandwidth view

We're going to compare two ways of updating a model from the information-theoretic view, and argue why SFT is much more reward dense and therefore sample efficient. This machine learning folklore is discussed in recent blogposts; see the post by [Thinking Machines](#) and the more formal treatment [here](#).

1. Vanilla policy gradient (REINFORCE-style)

- Sample a rollout from the current model.

- Score it with a reward signal.
- Push the policy toward higher-reward behavior.

2. Iterative SFT

- Repair the model's output until it passes our grader oracle.
- This repaired output is treated as "the correct answer."
- Train on it with normal supervised learning.

Both of these are trying to get the model to behave like some desired target policy.

Let's formalize LLM generation as an MDP. At time-step t , the state s_t is the prefix so far, and the action a_t is the next token. The chosen action (the next token) deterministically updates the state (the next token gets appended to the existing prefix). Let's assume the reward r_t is assigned per token and depends on some unknown parameters like human preferences or objectives. The reward induces an optimal policy $\pi^*(a|s)$. Our current model is a policy $\pi_\theta(a|s)$ which is called at each generation step. An answer of length T is a trajectory $\tau = (s_0, a_0, s_1, a_1, \dots, s_T)$. Training is about nudging π_θ towards the optimal policy π^* .

So when we talk about "information" here we mean: how many bits of information about π^* can the model see from one training example?

In basic policy gradient, the loop is:

1. Sample a full trajectory $\tau \sim \pi_\theta$.
2. Score that *entire* trajectory with a scalar reward $R(\tau)$
3. Form a gradient update $g = \nabla_\theta \log p_\theta(\tau) \cdot A(\tau)$

where $A(\tau)$ is an advantage term derived from the trajectory return $R(\tau)$, e.g. $A(\tau) = R(\tau) - b$ for some baseline b .

Given that the gradient g is what is observed by our model, an important consequence follows. One can make some arguments to conclude that in this expression, **all information about the learning task flows through the scalar $A(\tau)$** .

In particular, if we condition on the history $\mathcal{H}$ up to this point, then $\tau \perp \pi^* \mid \mathcal{H}$ and

$\nabla_{\theta} \log p_{\theta}(\tau) = \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t \mid s_t)$ is fully determined by the current model θ and gives no information about the target policy. The information the model can get from the gradient $I(g; \pi^* \mid \mathcal{H})$ is therefore upper-bounded by the entropy of this advantage scalar $H(A \mid \mathcal{H}, \tau)$. In our modelling setup, if we assume this advantage is calculated via a trajectory return $\sum_{t=0}^{T-1} \gamma^t r_t$ that collapses the rewards at each time-step to a single value, this vastly reduces the amount of information the model can get from one rollout.

Indeed, if one assumes some reasonable “finite resolution” of this return scalar (different reward sequences are collapsed into the same total return, the finite resolution of the scalar itself etc) this quite literally caps the amount of information obtainable from one rollout to a few handfuls of bits — $O(1)$ in the sequence length.

This gives a sense of why vanilla policy gradient is sample-hungry. You burn lots of rollouts to get a tiny trickle of information about what good behavior is, having collapsed a potentially rich set of rewards into a low bandwidth pipe into the model. It also hints at the credit assignment problem: the model just gets told “bad,” not “bad specifically at step t when you made the illegal tool call”.

Now let’s look at iterative SFT in the same frame. Assuming that our refinement evaluators are strong and will only accept perfect completions, we can treat our y^*y^* as a gold standard supervised training example that contains a denser reward higher-dimensional reward signal. Concretely, a reasonable approximation for this kind of supervised learning is to treat each state–action pair along this “approved” trajectory, $(s_t^*, a_t^*) \in y^*$ as a labeled datapoint following the optimal policy π^* .

Then we do standard next-token training, where the gradients become $\nabla_{\theta} \log p_{\theta}(y^* \mid x) = \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t^* \mid s_t^*)$. The model now gets dense feedback per step. If the model is still imperfect at this task, then many of those actions a_t^* – will differ from what π_{θ} would have done, which means they’re informative. The “surprise” of the full corrected answer under the current model is $-\sum_{t=0}^{T-1} \log \pi_{\theta}(a_t^* \mid s_t^*)$ and that scales with sequence length T . Given sequences on the order of thousands of tokens or above, one repaired trajectory can convey a much larger $O(T)$ number bits about π^* , not just a constant handful $O(1)$.

Sample efficiency in practice

Practically, this is why iSFT can get you most of the policy-shaping benefits you want from RL long before you run a single unstable RL step. You front-load dense information about π^* instead of trying to extract π^* from a trickle of 1-bit thumbs-ups.

In comparing sample efficiency, it's helpful to think about the number of LLM calls required to achieve similar levels of performance.

iSFT approach:

- Generate initial outputs from the model
- Use LLM calls (e.g., Gemini) to evaluate and provide feedback for refinement
- Iteratively refine outputs until they pass evaluation (typically 1-2 refinement rounds)
- Once we have a batch of perfect outputs y^*y^* , perform standard supervised fine-tuning

RL with LLM-as-a-judge approach:

- Sample rollouts from the current policy during training
- Use LLM judge to score *every single rollout* during the RL training loop
- Update policy based on scalar rewards
- Repeat for many thousands of training steps

The key difference: in iSFT, we make LLM calls *upfront* to create a high-quality dataset, then train efficiently via supervised learning. In RL with LLM-as-a-judge, we must make LLM calls *continuously throughout training* for every sampled trajectory.

Given our information-theoretic arguments above—that each iSFT example provides $O(T)$ bits of information while each RL rollout provides only $O(1)$ bits—we'd expect RL to require **orders of magnitude more LLM judge calls** to extract the same amount of information about π^* . If a typical sequence has $T \approx 1000$ tokens, the ratio can easily be an order of magnitude more LLM calls needed for RL to achieve comparable policy shaping. This makes iSFT not just more stable and easier to implement, but also far more *economical* in terms of compute and API costs when using powerful models as evaluators.

An alternative approach worth comparing to is "**poor man's RL**" (also known as rejection sampling). Like iSFT, this method also produces a supervised dataset from model outputs, but uses a different mechanism: instead of refining imperfect outputs, we simply have our model generate many attempts at the task and use a verifier to keep only the ones that pass. This also gives us a set of approved rollouts y^* which we can then use for standard SFT.

While both approaches produce supervised datasets of "good" examples, we expect iSFT to be more sample efficient and to scale much better as the task gets harder relative to the model ability. Rejection sampling can be extremely wasteful when the base model has a low success rate. If your model only succeeds 5% of the time, you need to generate 20 outputs to get one training example. iSFT instead takes failed attempts and repairs them through targeted refinement (typically a handful of rounds). This translates to far fewer LLM calls to build a dataset of the same size. Further, as the task gets harder and the rejection ratio becomes higher, we expect rejection sampling to scale exponentially worse.

Further, rejection sampling is limited to whatever your base model can already produce, as you're essentially filtering for its "lucky" outputs. iSFT actively improves outputs through refinement, which means the final dataset can contain solutions of higher quality than the base model would naturally generate. The refinement process can fix specific errors and inject corrections that pure sampling would struggle to discover.

Of course, neither iSFT nor poor man's RL produces perfect supervised data exactly from the π^* policy. Both rely on imperfect evaluators and produce approximations. But iSFT's refinement loop makes better use of each generation attempt and can systematically improve toward π^* , rather than hoping to stumble upon good examples through repeated sampling.

Future work

We have many directions to explore for improving iSFT further. One key area concerns the choice of evaluator model. Currently, we use a strong model as our evaluator, typically something like `gemini-2.5-pro` or `gpt-5` with maximum thinking budgets, which excels at providing high-quality feedback and refinements. However, there may be advantages to using a more *on-policy* model instead. If we're training Qwen but using Gemini's responses as our refined targets, we're introducing a potentially large KL divergence between Qwen's base policy and Gemini's generations. This distribution mismatch could make the learning task harder or less stable. In contrast, if we use Qwen itself (or a variant of it) to generate the feedback and refinements, the KL divergence would be much smaller; the refined outputs would be more naturally aligned with what the base model can learn, since they come from a similar policy distribution. This on-policy refinement approach could lead to faster convergence and better final performance.

For attribution in academic contexts, please cite this work as:

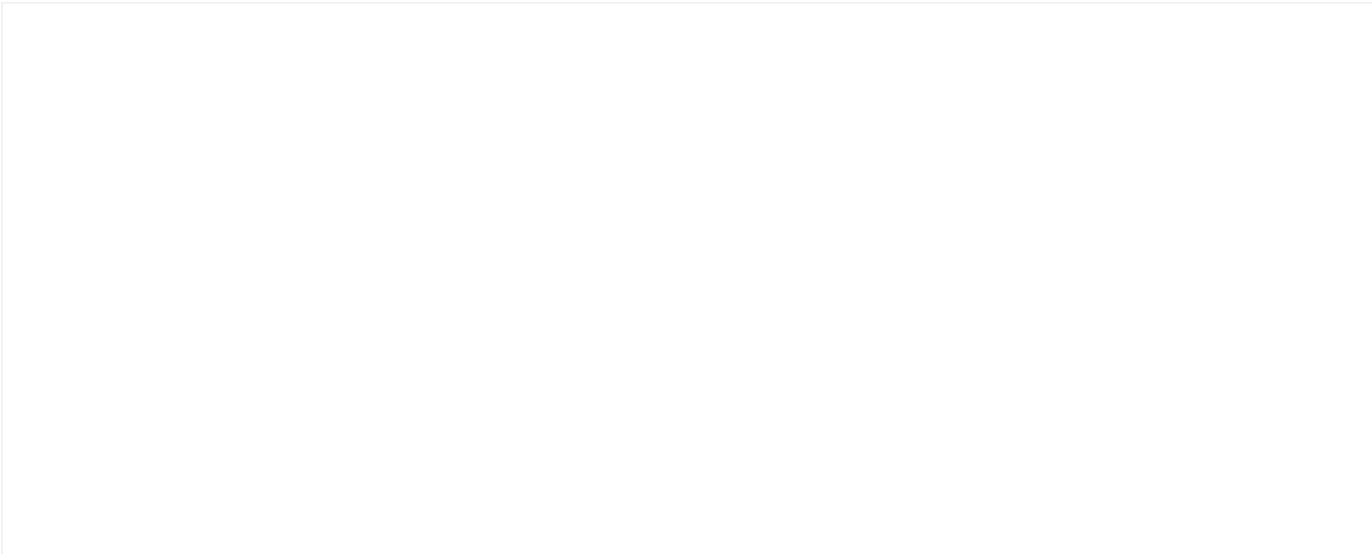
O'Neill, Charles. "Iterative SFT (iSFT): Dense Reward Learning." Baseten Research, October 15, 2025. <https://www.baseten.co/research/iterative-sft/>

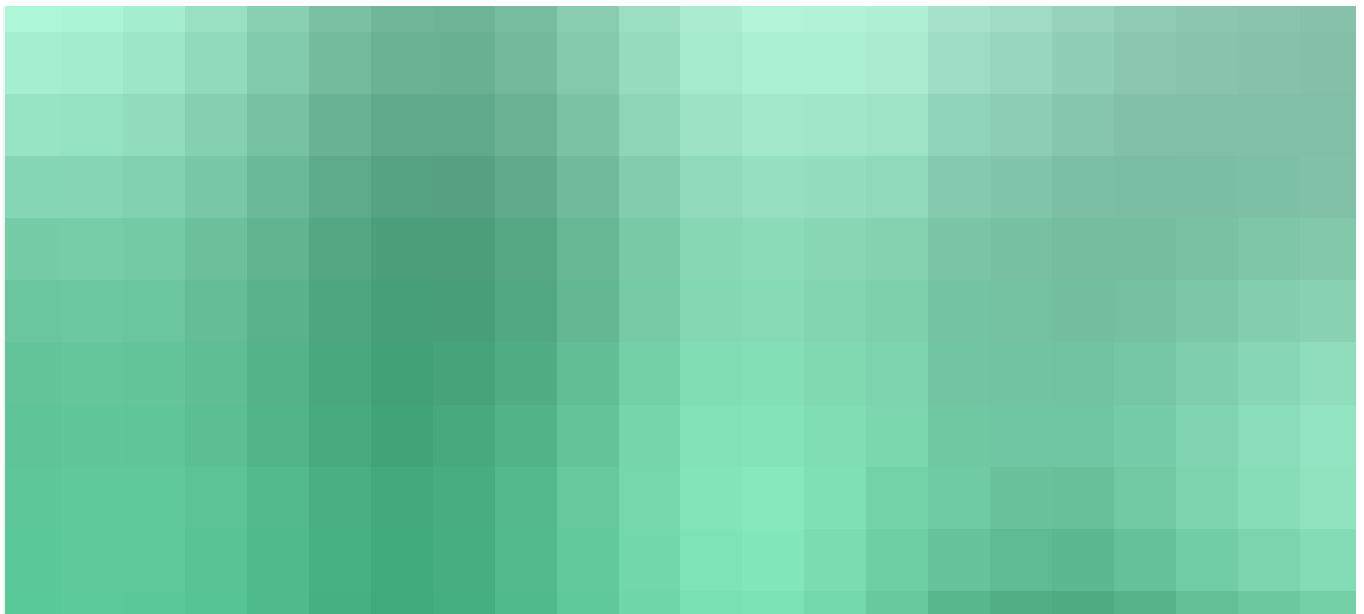
BibTeX citation

```
@misc{oneill2025isft,
  author      = {O'Neill, Charles},
  title       = {Iterative {SFT} ({iSFT}): Dense Reward Learning},
  year        = {2025},
  month       = oct,
  day         = {15},
  howpublished = {Baseten Research},
  url         = {https://www.baseten.co/research/iterative-sft/},
}
```

¹ Relation to DAgger. Conceptually, iSFT mirrors DAgger: the model acts on its own state distribution (prefixes), an expert provides corrections (grader+refiner), and we aggregate those corrections into the SFT corpus, eliminating covariate shift. Unlike classic DAgger, where the expert supplies per-state action labels from a fixed policy, our expert synthesizes a sequence-level correction y^* that may rewrite multiple steps. Because refinement can spend extra tokens (more chain-of-thought, tool calls, or structured reasoning) and we can also allow extra tokens at inference, y^* can exceed the performance of any fixed expert policy, effectively scaling quality with compute rather than capping it at the expert’s capabilities. This preserves DAgger’s stability while enabling super-expert targets via “compute-as-supervision”.

Other research





RESEARCH

Still: Amortized KV Cache Compaction in a Single Forward Pass

You can shrink a language model's KV cache by 200x, in a single forward pass, and it still answers correctly.



CHARLES O'NEILL • 4 others

RESEARCH

Post-training frontier legal agents with Baseten Research

Explore Baseten today

START DEPLOYING

TALK TO AN ENGINEER >

Product

- Dedicated Inference
- Model APIs
- Training
- Frontier Gateway

Baseten Inference Platform

- Model Runtimes
- Infrastructure
- Multi-cloud

Deployment options

- Cloud
- Self-hosted
- Hybrid

Embedded engineering

Forward deployed engineers

Modalities

- Transcription
- Image Generation
- Text-to-speech
- Large language models
- Compound AI
- Embeddings

Industries

- Enterprise
- Healthcare

Developer

- Model library
- Documentation
- Changelog

Resources

- Research
- Customers
- Blog
- Guides
- Events
- Partners
- Savings calculator
- Trust Center
- About us
- Startups
- Careers
- Contact us

Popular models

- GLM 5.2
- Kimi K2.7 Code
- DeepSeek V4
- GPT OSS 120B
- Whisper Large V3
- NVIDIA Nemotron 3 Ultra
- Explore all

Legal

- Terms and Conditions
- Privacy Policy
- Service Level Agreement

● ALL SYSTEMS NORMAL



© 2026 BASETEN

